601.445/601.645: Practical Cryptographic Systems Revised October 26, 2026

Assignment 2: JMessage — build it, negotiate it, break it

Instructor: Matthew Green *Due: 11:59 pm, November 9*

October 5, 2026; Revision 1, October 26, 2026 (see the changelog at the end). **Released:** Monday, October 5. **Client checkpoint (graded):** Friday, October 23, 11:59 pm. **Due:** Monday, November 9, 11:59 pm. **Review Lab 2:** week of November 9. This release contains a starter kit. A full reference implementation and exercises follow separately.

A Markdown copy of this handout (`handout.md`), the protocol specification, the starter kit and the agent instruction files are in the assignment repository: `https://github.com/matthewdgreen/practicalcrypto2026/tree/main/assignments/A2`

Read this handout together with the protocol specification (`specification.md`). The spec defines how the application will work and how it interoperates. This handout defines the work and assessment. JMessage deliberately contains insecure choices. Implement the assigned protocol, and discuss repairs in your written work.

0. Overview

JMessage is a small end-to-end encrypted messaging system built for this course. It comprises a server that relays messages without being able to read them, and clients that handle all encryption or decryption. Users register an account and upload an identity key, run a handshake with each other through the server to set up session keys, and then exchange encrypted text. This is an update of a past version of the assignment. The 2026 version adds some standard features of real messengers: a negotiated handshake that supports a post-quantum key exchange alongside a legacy non-PQC suite.

Your job is to implement the JMessage client from its specification, the way you would implement a real protocol from an RFC. The specification (`specification.md`) defines the wire format and behavior byte by byte. You won't have to code all the fiddly byte formatting pieces, though. We supply a starter kit that handles HTTP, JSON formatting/parsing, polling and the command line, and a local relay to test against. We also run a course server at `https://jmessage.practicalcrypto.org` (see §3), and the kit includes a local relay you can run yourself. Your job is just to write the cryptographic core. Your client must interoperate with ours and with your classmates'.

The protocol you implement includes:

- **accounts and an identity-key directory** on the server, with key fingerprints so users can check each other's keys out of band;
- **a three-message handshake** (HELLO, REPLY, FINISH) that agrees on fresh session keys with ephemeral X25519, signs the transcript with Ed25519 identity keys, and **negotiates one of two cipher suites**: JM2, a hybrid of X25519 and ML-KEM-768 with AES-GCM, or the legacy JM1, X25519 with AES-CBC and HMAC-SHA256;
- **encrypted text records** with sequence numbers, **delivery receipts**, and a **retransmission request** for records that arrive damaged.

It does not include attachments, persistent sessions, group chat, or recovery from network loss. I would love it if you made a graphical interface (coding agents are fine!). Creative and nice interfaces, bots, and personal agents built on top of your client are optional extra credit; see Part 1, "Optional: interfaces, bots and agents".

Your tasks, in order:

Part	What you do	When
0	Write the PKCS#7 padding functions yourself, by hand, before you use any coding agent on this assignment	first
1	Build the client: both suites, both roles, interoperating with the course `echo` user and the public checks	**checkpoint Fri Oct 23** (graded on this snapshot)
2	In a local sandbox we supply, play the server operator and show that the negotiation can be subverted	kit released Thu Oct 29
3	In the same sandbox, show that the legacy record layer leaks plaintext to an active attacker	kit released Thu Oct 29
4	Written questions: explain what failed, propose repairs, and say what they do and do not fix	**final Mon Nov 9**

Then, the week of November 9, a 15-minute in-person **Review Lab** on your own code. Coding agents are allowed everywhere except Part 0, and the kit includes instructions that make them teach rather than just produce; the lab and the quizzes measure what you understood, not what your agent wrote.

1. How JMessage works

JMessage has the same basic architecture as Signal, WhatsApp or iMessage. A central **server** stores user accounts, a directory of public keys, and a mailbox for each user. **Clients** send each other encrypted messages by dropping them in the recipient's mailbox. The server relays every message but should not be able to read or alter any of them: all encryption and authentication happens in the clients. JMessage treats the server as **untrusted**, and so should you.

A conversation in JMessage works like this:

- **Identity.** Each client generates a long-term Ed25519 signing key and publishes the public half in the server's directory. The public key is how other users recognize you, and its fingerprint is how a careful user can check that the server hasn't substituted a different key.
- **Handshake.** Before two users can exchange messages, their clients run a short handshake through the server. The handshake agrees on fresh session keys using ephemeral Diffie–Hellman (X25519), optionally combined with the post-quantum KEM ML-KEM-768. Both sides sign the handshake transcript with their identity keys, so each knows who is on the other end.

- **Negotiation.** JMessage supports two *cipher suites*. The modern suite, JM2, is a hybrid of X25519 and ML-KEM with AES-GCM for messages. The legacy suite, JM1, uses X25519 only, with AES-CBC and HMAC. The two clients negotiate which suite to use, as TLS does. Real protocols carry legacy options like this for compatibility with older software, and how they are negotiated turns out to matter.
- **Records.** After the handshake, each text message travels as an encrypted *record* under the session keys, with a sequence number. The recipient acknowledges it with a receipt, or asks for a retransmission if the record arrives damaged.

JMessage has been part of this course for several years; the 2026 version replaces its predecessor's single-message encryption with the handshake and suite negotiation described above. It is realistic enough that the lessons transfer to deployed systems. It is also **deliberately flawed**. Some of its design decisions are ones that real protocols made and later regretted. You will first build a faithful client, then examine where the design fails and how to repair it. The protocol specification (`specification.md`) is the authoritative description; this section is only an overview.

2. What you should learn

By the end of this assignment you should be able to:

- Explain which bytes a signature authenticates and how a public key becomes associated with a person.
- Trace both participants' computation of the same session keys, including the different roles of key generation, encapsulation, and decapsulation.
- Explain directional keys, sequence numbers, IVs, nonces, and the state that must survive between messages.
- Distinguish confidentiality, message authentication, peer authentication, delivery indications, and key confirmation.
- Evaluate how negotiation policy and error handling affect security, propose a repair, and state what that repair does and does not guarantee.

You are building a small protocol implementation, not a production messaging application. Attachments, persistent sessions, automatic network-loss recovery, and implementing cryptographic primitives are outside the scope. A user interface, a bot, or a personal agent is not required, but each is encouraged as optional extra-credit work built on top of your protocol core (see Part 1, "Optional: interfaces, bots and agents").

3. Environment and supplied scaffolding

Use Python, as required by the syllabus. An alternative language requires instructor permission before beginning, and because grading drives your client through the Python interface below, you would also need to supply a Python adapter for it. Use Python 3.12 or newer and the starter kit's pinned `requirements.txt` (`cryptography` 49.0.0 and its Python dependencies). The starter-kit guide (`starter/README.md`) gives setup commands and includes an ML-KEM availability check. A package version alone does not establish backend support. Do not implement ML-KEM, X25519, Ed25519, AES, HMAC, or HKDF yourself.

The starter kit supplies:

Supplied component	Purpose
Account and HTTP helpers	Registration, login, API-key handling, key upload and lookup
Command-line driver	Input parsing, base64 envelopes, serialized sending, ordered polling, and display
Local identity storage	Generating/loading the client's Ed25519 key with library calls
Encoding helpers	Integer and vector encoding, bounded readers, strict base64 and UTF-8 handling
Message formats (`jmessage_support/messages.py`)	Encoders and parsers for the seven message bodies, with §3's structural checks; constants for types, suites, reasons and field sizes
Test driver	In-memory transport, an injected clock, and readable conformance results
Submission template	Stubs for the protocol core and written-work files

You implement the protocol state machine, transcript construction and signature checks, KEM/DH and key-derivation orchestration, record protection and validation, and the warmup below. The supplied message formats pack and unpack bytes; they do not construct transcripts, decide which data is authenticated, derive keys, or decide whether a message is acceptable. These decisions remain visible in your code, and you should still be able to write out a HELLO byte by byte.

The course server is at `https://jmessage.practicalcrypto.org`. Its shared `echo` account replies to every message with the same text, in either suite; its identity-key fingerprint is `BE 9F 94 00 02 63 66 86 A2 7E`, so you can check that your fingerprint code and the server's directory agree. The `echo` account is for convenience and interoperability practice. Grading runs locally against controlled peers; course-server availability and network latency do not affect the grade. Authenticated peer keys are supplied for controlled security exercises, as described in specification §5.

Protocol-core interface for the Python starter kit

Submit `protocol.py` exporting a `Client` class with this interface:

```
Client(username, identity_private_key, lookup_peer_key, emit, now,
       *, jm1_only=False)
client.start_handshake(peer)
client.send_text(peer, text)
client.receive(sender, recipient, body)
client.check_timeouts()
client.fingerprint(peer=None)
client.reset_peer_trust(peer)
client.transport_failed(peer=None)
```

`check_timeouts`, `reset_peer_trust`, and `transport_failed` implement the optional robustness practices in specification Appendix B. They must exist, but they may be no-ops; they are not graded.

`identity_private_key` is a raw 32-byte Ed25519 private key. `lookup_peer_key(username)` returns a raw 32-byte public key. It is backed by the directory or the authenticated exercise fixture. If the user has no usable published key, it raises `jmessage_support.transport.PeerKeyUnavailable` (a `LookupError`); handle this by reporting an error and not proceeding with that handshake (specification §5). Network failures raise `TransportError` instead, and the driver stops. `now()` returns monotonic seconds. `emit(event)` is a synchronous callback. The starter kit owns HTTP, base64, polling, and clock advancement; `body` is already decoded bytes. Methods other than `fingerprint` return `None` and emit zero or more events in processing order. `fingerprint` returns the formatted fingerprint string; `peer=None` means the local user.

Event dictionary	**Meaning**
`{"type": "send", "to": peer, "body": bytes}`	Send a complete binary protocol body
`{"type": "established", "peer": peer, "sid": bytes, "suite": int}`	A local session became established
`{"type": "message", "peer": peer, "sid": bytes, "seq": int, "text": str}`	Deliver authenticated, valid text
`{"type": "receipt", "peer": peer, "sid": bytes, "seq": int}`	An accepted receipt arrived
`{"type": "warning", "peer": peer, "message": str}`	A local warning, such as a RESEND request
`{"type": "error", "peer": peer, "message": str}`	A local operation or handshake failed

A successful `emit` of a send event means the transport accepted the send; an exception means failed or uncertain delivery. Apply specification §4.6. The callback does not reenter the client: incoming messages are processed by later `receive` calls. Error/warning text is not graded. Malformed peer input must follow the specified discard or failure behavior rather than escape as an uncaught exception. Cryptographic processing completes before an event that exposes plaintext. `start_handshake` explicitly begins a fresh handshake even if another session exists. `send_text` queues text if necessary, as in §6.9. `reset_peer_trust` is an explicit local user action, never an automatic response to a peer message. The driver calls `transport_failed(peer)` after a transport failure affecting that peer, or `transport_failed()` after an uncertain receive, then stops. The optional hook may abandon affected sessions as recommended in Appendix B, but the driver does not depend on it to continue safely. Error events for an unknown affected peer may use `"peer": None`.

The supplied `client.py` exposes these operations; its terminal syntax and launch examples are in the starter-kit guide. The autograder uses this Python interface, so terminal formatting is not graded. Students who implement their own interface must retain this adapter. A loopback-only local relay lets you practice with two ordinary clients without waiting for the course server; it contains no reference client or exercise bots.

4. Parts and proposed points

Part	Work	Points
0	Independent padding warmup	5
1	Interoperable client, both roles and suites	25
1b	Hand-traced handshake, in `design.md`	5
2	Controlled negotiation-security exercise	20
3	Controlled record-layer-security exercise	20
4	Written explanation and repair analysis	25
	Total	**100**
	Extra credit (Part 1, an interface, bot or agent built on your client)	up to 6
	Extra credit (Part 4, questions 7–9)	up to 6

Part 0 — Write a small component yourself

Before asking a coding agent for an implementation, write `pkcs7_pad(data: bytes, block_size: int = 16) -> bytes` and `pkcs7_unpad(data: bytes, block_size: int = 16) -> bytes` in `padding.py`. Support block sizes 1 through 255. Padding always appends at least one byte, including a full block when the input is aligned. Unpadding raises `ValueError` for an invalid block size, empty input, non-block-aligned input, or invalid padding. Use these functions for JM1.

You may consult lecture notes and language documentation. Write these two functions and your initial examples without generated code or a library padding helper. After that first attempt, AI feedback and debugging help are permitted; retain the initial version as `padding_first_attempt.py` and describe revisions in your AI usage note. This bounded exception to take-home AI permission applies only to these functions and the initial examples, not the rest of the client.

Include examples for empty input, an aligned input, an input one byte short of a block, and invalid padding. Explain in your own words why padding is always added. Implementing AES or CBC itself is not part of the warmup.

Part 1 — Build the client

Implement specification §§5–7 behind the supplied driver. Work in this order: padding and record protection; JM1 handshake and messaging; JM2 key agreement; then error handling and state transitions. Start with the JM1 compatibility option while developing, but the finished client's default must offer both suites as specified.

Write these yourself. As with the Part 0 padding, three pieces of the client must be written by you, not generated: `receive` (the dispatch on message type and the state table from specification §6.9 that it enforces), `send_text` (session selection and queueing), and at least one handshake handler end to end, either the REJECT/fallback handler or the FINISH handler. A coding agent may explain the spec, review your draft, and help you debug it, but may not author these. The remaining handlers may be agent-written if you choose, provided you can explain them. Say in `ai-usage.md` which of these you wrote and what help you had. This is an assignment rule; the agent instruction file treats it as one.

The reason is the same as for the padding: the state machine is the largest part of the review lab, and in a trial run a student who delegated it finished with a correct client of which they had written no part.

Part 1 and Part 1b are graded on your checkpoint submission (Friday, October 23), because the exercise kit released after the checkpoint contains a working reference client. The public eligibility checks are available from release, so you can see where you stand before submitting.

You may keep fixing your client after the checkpoint. The final submission's client is also run, and fixes to checkpoint failures recover up to 5 of the lost Part 1 points, provided `design.md` explains each fix in your own words. Keep the checkpoint snapshot in `checkpoint/`.

Part 1b — Hand-traced handshake (5 points, in `design.md`)

Take the JM1 handshake in specification Appendix A.1 and annotate it by hand, in `design.md`, byte by byte:

- the HELLO: mark each field (type, `sid`, suite count and suites, `X_I`, the empty `ek_I` vector) with its offset and length;
- the REPLY: the same for `REPLY_core`, then `sig_R`;
- `TH_R`: write out, in order, exactly which byte strings are hashed, show where `str8(alice)` and `str8(bob)` appear, and say which copy of the HELLO each side uses and why the initiator's recomputation matches the responder's;
- one DATA record in each direction: mark the IV and ciphertext, say which key encrypts and which key MACs it, and write out the exact bytes under the MAC.

Half a page to a page. Hex and arrows are fine; it does not need to be pretty. Write it yourself: a coding agent may answer questions and check your annotation against the vectors afterwards, but the annotation is yours, like the padding, and `ai-usage.md` should say so. It is due with the client checkpoint and graded with it. It is also the Review Lab's starting point: the first thing the interviewer will put in front of you is your own trace, so the time it takes is time you would spend preparing anyway. A trace produced by an agent is easy to recognize and hard to defend.

Optional: interfaces, bots and agents (extra credit, up to 6 points)

The supplied `client.py` prints JSON events. You may build something real on top of your client: a **user interface** (a terminal UI, a desktop window, or a local web page), a **bot** (an account that answers messages on its own: an auto-responder, a game, a summarizer), or a **personal agent** (software that reads your messages and acts for you, for example by drafting or sending replies, possibly backed by a language model). All of these are glue in the sense of the agent instructions, so your coding agent may build all of it, and we would like to see what you come up with.

Two rules apply to all of them. First, whatever you build sits **on top of** the `Client` interface in `protocol.py`: it calls `start_handshake`, `send_text` and `fingerprint`, and it consumes the events the core emits. It must not touch keys, transcripts or records, and it must not read or display message text before the core has emitted a `message` event for it. Keep `protocol.py` importable by the autograder unchanged. Second, the supplied driver

runs every protocol call on one thread; anything with its own event loop must hand actions back to that loop rather than call `Client` from another thread (the starter-kit guide has a note on this).

For a user interface, credit is for one that **tells the user the truth about their security state**, not for visual polish alone. Full credit requires all of:

- each conversation shows which suite its current session negotiated, with the legacy suite visibly distinguished from the hybrid one;
- the peer's fingerprint is shown, and the user can mark it as verified by hand, with that mark displayed afterwards;
- `warning` and `error` events (for example a RESEND request or a failed handshake) are shown in the conversation, not only in a log;
- a change in a peer's directory key is shown prominently and clears the verified mark.

For a bot or a personal agent, credit is for the thinking as much as the code: anything that reads messages and acts on them is a new endpoint, and the assignment's encryption protects nothing past the endpoint. Alongside the code, answer these in a page or so in `design.md`:

- *Security.* What can a message from a stranger make your bot or agent do? What data can it reach, and what can it send out? If it is backed by a language model, what happens when a message contains instructions addressed to the model? State the threat model you designed for, and what you did not defend against. (Part 4's extra-credit questions are about exactly this.)
- *Data privacy.* What leaves the user's machine, to whom, and in what form? If you call a hosted model, say which plaintext goes to the provider and what the user is told about that. What is logged, and for how long?
- *UX.* How does the user know the bot or agent acted on their behalf? Does it confirm before sending anything? How does the user see what it saw, correct it, or turn it off?

Rules for bots and agents: run them only against the course server or the local relay, from accounts you own; never make one message another student's account without that student's consent (your own second account and `echo` are always fair game). If you use a hosted model, use your own API key, never put it in the submission, and disclose the use in `ai-usage.md`.

Submit the code with your client, and link two or three screenshots or a short recording from `design.md`. After Part 2 you will be asked whether your interface, bot or agent would have told you what happened; keep that in mind. The cap is 6 points for this subsection in total, however many of these you build; one thing done well beats three done hastily.

Parts 2 and 3 — Controlled security exercises

These parts examine negotiation downgrade and record-layer error handling in the supplied local teaching environment. The exercise kit and its submission interface will be released after the client checkpoint. They will specify the required result, evidence to submit, execution limits, and partial-credit criteria. The kit is released separately.

Submit the exercise work and a concise explanation of the security property that failed, the assumptions required, and the evidence supporting your conclusion. The written explanation is assessed separately from successful execution. Use the course-provided local environment for these exercises.

Part 4 — Explain and repair

Answer the following in `written.md`, normally one or two paragraphs per item. Refer to the relevant functions in your submission where useful.

1. What establishes the association between a username and an identity key? What does a valid signature establish when that association is untrusted?
2. Trace the initiator's and responder's inputs to the JM2 key schedule. Explain why they agree and why the two traffic directions use different keys.
3. Which negotiation data does each signature cover? Explain why checking one handshake transcript does not by itself establish the security of the application's broader negotiation policy.
4. Explain the distinct receive outcomes for JM1 and JM2 and their security implications. Evaluate a record-layer repair and describe the acceptance and rejection behavior it should have.
5. Propose a negotiation-policy repair. State the property restored, its compatibility cost, and a limitation it does not address.
6. Does adding ML-KEM make this entire protocol post-quantum secure? Distinguish key agreement from identity authentication, and explain what FINISH and RECEIPT do and do not confirm.

Extra credit (up to 6 points): assistants at the endpoint

Suppose a JMessage client includes an AI assistant that reads incoming messages, summarizes conversations, and can send replies on the user's behalf. Its model runs either on the user's device or on a remote inference service inside a trusted execution environment (TEE) whose code is publicly attested. Answer each question in two or three paragraphs. No code is required, and you should not test any of this against real services or other students.

7. End-to-end encryption is usually described as protecting messages "from everyone except the endpoints." Where is the endpoint in this design? Use the lethal-trifecta framing (access to private data, exposure to untrusted content, and a channel to send data out) to explain why a message from an authenticated sender can still cause the assistant to disclose another conversation. Which JMessage guarantees still hold, and which ones stop mattering?
8. What does a TEE with remote attestation actually guarantee here, and to whom? Compare the threat it addresses with the threat in question 7. Use a deployed design as your example, such as Meta's Private Processing for WhatsApp or Apple's Private Cloud Compute.
9. If many users run such assistants, explain at the level of system design how a single message could spread from assistant to assistant, as in the Morris II study of self-replicating prompts (Cohen, Bitton and Nassi, 2024). Evaluate two mitigations and state what each one costs in usefulness. Examples: user confirmation before any send, per-conversation context isolation, tracking where data came from, or plan-then-execute designs such as CaMeL. Is any of these mitigations a cryptographic property?

5. Proposed release and checkpoint sequence

Date	Milestone
Before the client checkpoint	Release the specification, handout, working starter kit, and public eligibility checks together
Wed Oct 7	Midterm exam (covers material through Sep 30; A2 is not on it)
Mon Oct 12	ML-KEM lecture; finish the hybrid (JM2) portion after this lecture if needed
Fri Oct 9	Suggested, ungraded progress target: padding and JM1 records
Fri Oct 23, 11:59 pm Eastern	Client checkpoint, including both suites; Part 1 is graded on it
Thu Oct 29, 9:00 am Eastern	Last late checkpoint accepted; reference-client and exercise-kit release
Mon Nov 9, 11:59 pm Eastern	Final submission
Week of Nov 9	Review Lab 2

The KEM API abstraction is sufficient to begin JM2 before the ML-KEM lecture; implementing lattice arithmetic is never required. Checkpoint and kit-release dates must be confirmed when the assignment is released. If the starter kit is delayed, staff must revise the checkpoint schedule rather than shorten the announced working period. The syllabus's late-hours policy applies to both the checkpoint and the final submission. A late checkpoint is accepted only until the kit is released (Thursday, October 29, 9:00 am); after that it cannot be submitted, because the reference client is public.

6. Review lab and eligibility

The 15-minute review is graded separately under the syllabus. Its focus is your understanding of these regions:

- Transcript construction, identity-key selection, and signature checks.
- Shared-secret derivation, directional keys, nonce/IV handling, and counters.
- Padding and authenticated record processing, including failure behavior.
- Your explanation of the security exercises and proposed repairs.

Expect to trace a short example, explain an invariant, and discuss a small change to your code. The session opens with your own hand-traced handshake from `design.md` (Part 1b) on the screen, and the questions begin from it. Networking boilerplate and terminal formatting are not review-critical. The review rubric assesses tracing behavior, explaining why it works, and reasoning about a modification; the staff's detailed rubric will be published with the kit.

For this assignment, **passing the autograder for review eligibility** means passing the published `review_eligible` group: the submission imports in the course environment; the padding warmup passes its ordinary boundary cases; and the client completes an ordinary JM1 handshake and text exchange in both roles. The required written files and AI usage note must also be present. All eligibility checks are public and runnable locally. Hidden

edge cases, optional retransmission, shared-server access, and success on Parts 2–3 cannot block a review signup. Other checks, including JM2, still determine assignment points and remain within the review's announced conceptual scope.

Run `python run_checks.py review_eligible` from the starter directory. Its current exchange checks run two copies of your client, so consistent mistakes can pass; this is a minimum progress check, not proof of conformance or security. The separate `client_smoke` group also exercises JM2. It can use an independent staff-supplied peer once that reference implementation is available. Fuller conformance checks determine assignment points separately from eligibility.

Students who used an agent must also complete the separate anonymous-report attendance requirement in §7 before signing up. Students who used no agent are exempt from that reporting requirement.

7. Submission and AI use

Submit a directory containing:

```
protocol.py
padding.py
padding_first_attempt.py
design.md
written.md
ai-usage.md
checkpoint/                 # client checkpoint source snapshot
exercises/                  # Parts 2-3; exact contract follows with the kit
```

Include any supporting modules needed to import your client. Do not submit account passwords, API keys, or private keys from actual course-server use. `design.md` briefly maps the review-critical functions and records what you tested and what remains incomplete. Include your warmup examples here, and the hand-traced handshake of Part 1b.

For the client checkpoint, the supplied `package_submission.py` builds an archive of your client source and written files, excluding local credentials and virtual environments. It does not package the later security exercises. Preserve your submitted source in `checkpoint/`; the exercise kit will supply the full final packaging instructions.

Outside the initial Part 0 attempt, AI assistance is permitted and encouraged with the disclosure required by the syllabus. In `ai-usage.md`, state which tools you used, what you delegated, what you checked yourself, and one thing you learned or corrected. State whether you completed the initial warmup without generated code; disclose any deviation rather than making a false statement. State which of `receive`, `send_text` and your chosen handshake handler you wrote yourself (Part 1). If you used the course agent instructions, also state whether you completed the readiness check they describe (the six questions, the live change, and the walkthroughs of agent-written code), and which parts you skipped; skipping is allowed, misstating it is not. Using no AI is fine and should also be stated.

When using an agent on a review-critical region, ask it to show and explain the proposed code, then explain the relevant invariant yourself before moving on. You remain responsible for understanding your submission. The optional course agent instructions will reinforce this practice; the assignment does not require a particular editor or AI tool.

Anonymous agent report and review-lab sign-up

As announced in the A1 reporting handout, **students who used an agent must submit an anonymous agent report and an attendance proof before signing up for Review Lab 2. Students who used no agent are exempt from both.** State no-agent use in `ai-usage.md` and follow the staff's no-agent signup route; do not create a fictitious report or proof. The report's content is ungraded.

The two documents serve different purposes. `ai-usage.md` is your named disclosure in the programming submission. `agent-report.md` is the agent's short account of how the tutoring worked, including its own mistakes and limits of context. Even light agent use warrants a brief, honest report. Review and correct it, remove identifying details, and explicitly approve it before it is sent through the separate anonymous channel. Keep the report, private contribution logs, deposit receipts, and report tags out of your named submission.

The starter includes `tools/anon-report.py`; `AGENTS.md` explains the process. Reuse your registered A1 channel key. If you do not have one, register a public key through the staff-announced route; never submit the private key. Use the official roster that includes your key and wait for staff to open A2 deposits. After depositing via Tor with `--assignment A2`, wait for the official `tags-A2.json` snapshot and generate `attendance.json` locally. Submit that proof only to the separate A2 attendance assignment, following staff's signup instructions. The proof establishes participation without identifying which anonymous report is yours. Do not include it in the programming archive.

Reporting dates: register your public key by **October 23** if you have not already registered one, and deposit your report by **November 8**. Staff will publish the tags snapshot on **November 9**. Staff will publish the updated roster and announce the attendance submission and no-agent signup routes before they are needed. If a missing key, snapshot, or submission route prevents completion, contact staff privately for help; do not bypass the reporting tool's privacy threshold.

Changelog

- **Revision 1 (October 26, 2026).** The reference client and the Parts 2–3 exercise kit are released on Thursday, October 29 at 9:00 am instead of Monday, October 26. Late client checkpoints remain accepted until that release. The final submission moves from Friday, November 6 to Monday, November 9 at 11:59 pm, so the working period after the kit is unchanged. Nothing else changed.

Authorship and AI usage note *(in the same format this course asks of you).* This handout, the protocol specification, the starter kit and the course server were written by **Matthew Green** with **Claude** (Anthropic's Fable 5 model, via Claude Code). Delegated to the AI: drafting the specification and this text from the 2024 JMessage materials, the starter kit and its tests, the staff reference implementation and conformance suite, the server deployment, and the LaTeX conversion. Done by the human: the assignment's design — the move to a negotiated handshake with a post-quantum suite, what is examined and why, the "cryptography is not glue" rule for agents, and every grading decision — plus review

and editing of every section. One thing we caught: the first draft of the specification left a dozen behaviors ambiguous (what a client does with a REPLY from the wrong peer, when `send_text` starts a handshake, what "session identifier in use" means). None of us noticed until a second, independently written client was built from the specification alone and compared against the first; every ambiguity became a sentence in version 1.1. Build two of anything before you trust the description of it.